

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia (TUPaD)

Asignatura: Programación 2

TRABAJO PRÁCTICO INTEGRADOR:

FOOD STORE - SISTEMA DE GESTIÓN DE PEDIDOS DE COMIDA

Estudiante: Matías Ferreyra

Fecha de Entrega: 01/10/2026

Repositorio GitHub: [Ir a GitHub](#)

Video Demostrativo: [Video Youtube](#)

ÍNDICE GENERAL

Índice	Pág. 2
Marco Teórico	Pág. 3
Decisiones Técnicas y Arquitectura	Pág. 4
Arquitectura en Capas y Estructura de Paquetes	
Justificación del Almacenamiento Mixto	
Respeto Estricto a la Direccionalidad del UML	
Modelo de Datos Relacional	Pág. 6
Tablas y Relaciones	
Script DDL de Creación	
Dificultades y Soluciones	Pág. 7
Capturas del Sistema Funcionando	Pág. 8
Bibliografía y Webgrafía	Pág. 17

MARCO TEÓRICO

El proyecto FoodStore ha sido desarrollado bajo el paradigma de Programación Orientada a Objetos (POO) en Java 21, aplicando estándares de diseño orientados a aplicaciones de arquitectura limpia, robusta y escalable.

Java 21 y Principios POO

Encapsulamiento: Ocultamiento de la representación interna del objeto garantizando la integridad de los datos mediante métodos getters y setters.

Herencia: Implementación de la clase abstracta Base (id, eliminado, createdAt), la cual centraliza los atributos comunes para todas las entidades del dominio (Categoria, Producto, Usuario, Pedido, DetallePedido).

Polimorfismo e Interfases: Uso de extensiones de la interfaz Calculable para el cómputo dinámico de totales, y de la interfaz genérica GenericDAO<T> para estandarizar el comportamiento CRUD de la capa de persistencia.

Java Database Connectivity (JDBC)

JDBC constituye la API estándar de Java para la ejecución de operaciones SQL directas. En esta solución no se utilizan frameworks ni gestores de dependencias. Las bases del acceso JDBC utilizadas son:

Connection: Gestión de la sesión con la base de datos MySQL 8. Manejo explícito de transacciones mediante setAutoCommit(false), commit() y rollback().

PreparedStatement: Ejecución de consultas parametrizadas para evitar ataques de Inyección SQL y optimizar el rendimiento de compilación en el motor de base de datos.

ResultSet: Mapeo iterativo del conjunto de resultados devueltos por la base hacia objetos de la aplicación.

Archivos de Configuración XML y Parsing DOM

Siguiendo las especificaciones de Jakarta Persistence, la configuración de conexión se desacopla del código Java utilizando un archivo src/META/persistence.xml. El archivo se procesa utilizando la API nativa de Java DOM Parser (javax.xml.parsers.DocumentBuilderFactory), garantizando la portabilidad del sistema sin requerir librerías de terceros.

DECISIONES TÉCNICAS Y ARQUITECTURA

Estructura de Paquetes

El sistema adopta una Arquitectura en Capas rigurosa donde la lógica del menú no conoce SQL, y el acceso a datos no conoce la interfaz de usuario.

META: Contiene persistence.xml para desacoplar las credenciales de la base de datos sin usar librerías externas.

Main: Contiene únicamente el método main() que arranca el sistema e invoca al menú principal.

config_: Gestiona la lectura del archivo XML mediante la API DOM nativa de Java y administra Connection JDBC.

entities: Contiene el modelo de objetos que extienden de la clase abstracta Base (id, eliminado, createdAt). Respetando estrictamente las relaciones unidireccionales del UML sin añadir referencias inversas.

enums: Agrupa las constantes Rol (ADMIN, USUARIO), Estado (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO) y FormaPago (TARJETA, TRANSFERENCIA, EFECTIVO).

exception: Define las excepciones propias (PedidoInválidoException, ProductoInválidoException, StockInsuficienteException) para evitar que el programa se interrumpa por errores o stack traces no controlados.

interfaces: Define los contratos comunes del sistema, donde está ubicado la interface Calculable

dao: Implementaciones de acceso a base de datos relacional para Categoria y Producto mediante PreparedStatement y ResultSet. Es el único paquete autorizado para escribir SQL.

repository: Administra las colecciones dinámicas en memoria para las entidades Usuario, Pedido y DetallePedido. Mediante una clase madre llamada UsuarioRepository.

services: Contiene los casos de uso, las reglas de negocio (validación de precios, stock, unicidad de mail, etc.) y la coordinación de transacciones entre memoria y base de datos.

ui: Gestor de vistas por consola y lectura de entradas del teclado. Muestra el origen de los datos de cada opción y captura excepciones para informar al usuario.

Justificación del Almacenamiento Mixto

Entidad	Almacenamiento	Justificación Técnica
Categoría	Base de Datos Relacional	Información primaria y persistente del catálogo comercial. Se almacenan en la tabla “Categorías” en la BD
Producto	Base de Datos Relacional	Requiere consistencia en precio y stock transaccional compartido. Se almacenan en la tabla “Productos” en la BD
Usuario	Memoria (List<Usuario>)	Datos temporales de sesión u operacionales en memoria para la simulación. Se almacenan en la lista estática “usuarios” de UsuarioRepository.
Pedido y Detalle	Memoria (List<Pedido>)	Viven acoplados al ciclo de vida en memoria del Usuario (asociación unidireccional) Se almacenan en la lista de pedidos de cada usuario de la lista estática “usuarios” de UsuarioRepository.

Transaccionalidad en el punto de contacto

Al crear un Pedido (en memoria), los Productos que lo componen deben descontar su stock en la Base de Datos. Para evitar inconsistencias, el servicio inicia una transacción JDBC activa (setAutoCommit(false)). Si el stock de alguno de los productos es insuficiente o la conexión falla, se ejecuta un rollback(), se lanza una excepción propia (StockInsuficienteException o ConexionFallidaException) y el pedido no se registra ni en memoria ni en la base.

Respeto a la Direccionalidad del UML

Conforme a las pautas del trabajo práctico, ninguna relación es bidireccional:

- La clase Producto no tiene un atributo categoria ni categoria_id. La navegación Categoría -> Productos se logra mediante una consulta parametrizada en el ProductoDAO: SELECT * FROM producto WHERE categoria_id = ?.
- La clase Pedido no posee un atributo usuario o usuario_id. Usuario conoce sus pedidos, pero un pedido no conoce al usuario que lo creo.
- La clase DetallePedido conoce su Producto, pero Producto no guarda listas de detalles.

MODELO DE DATOS RELACIONAL

El almacenamiento persistente consta exclusivamente de dos tablas. Queda totalmente excluida la creación de tablas para usuarios o pedidos.

Estructura de Tablas

Tabla categorias:

id: BIGINT (PK, Auto_Increment, NOT NULL).

nombre: VARCHAR(50) (NOT NULL).

descripcion: VARCHAR(50) (NOT NULL).

eliminado: BOOLEAN (Default FALSE para baja lógica, NOT NULL).

created_at: TIMESTAMP(Default current_timestamp, NOT NULL).

Tabla productos:

id: BIGINT (PK, Auto_Increment, NOT NULL).

nombre: VARCHAR(50) (NOT NULL).

precio: DOUBLE (NOT NULL).

descripcion: VARCHAR(50) (NOT NULL).

stock: INT (NOT NULL).

imagen: VARCHAR(50)(NOT NULL).

disponible: BOOLEAN (Default TRUE).

eliminado: BOOLEAN (Default FALSE, NOT NULL).

created_at: TIMESTAMP(Default current_timestamp, NOT NULL).

categoria_id: BIGINT (FK -> categorias(id)).

DIFICULTADES Y SOLUCIONES

Durante el desarrollo del proyecto FoodStore me encontré con varios problemas prácticos al momento de programar. A continuación, explico los principales obstáculos que tuve y cómo los fui resolviendo:

- En la base de datos, la tabla producto necesita tener una columna categoria_id para saber a qué categoría pertenece cada producto. Sin embargo, en el diagrama UML que se dio, la clase Producto no tiene ningún atributo para guardar la categoría ni su ID. Para resolver esto, se decidió que la clase Producto en Java se mantenga tal cual pide el UML (sin guardar el id de categoría). El categoria_id solo se pide por teclado cuando el usuario crea un producto y viaja directamente como un parámetro al método del DAO (save(producto, idCategoria)). De esta forma, SQL guarda la clave foránea en la base de datos, pero nuestro código Java no rompe la regla del UML.
- Los usuarios y pedidos se guardan en memoria, pero el stock de los productos está en la base de datos. Cuando un cliente arma un pedido con varios productos, se cuenta con el riesgo de que el stock del primer producto se descuente bien, pero que al llegar al tercero no alcance el stock. Si eso pasaba, nos quedaban datos a medias: productos descontados en la base de datos, pero el pedido cancelado en la memoria. Para evitar esto se usó transacciones en JDBC. Antes de modificar el stock desactivamos el guardado automático con `connection.setAutoCommit(false)`. Vamos probando descontar cada producto y, si alguno falla o no tiene stock suficiente, ejecutamos un `connection.rollback()`. Así se asegura de que se descuente el stock de todos los productos juntos o directamente no se descuente nada, evitando que la base de datos quede desactualizada.
- Cuando fallaba la conexión a la base de datos o se escribía mal una consulta SQL, Java lanzaba un error gigante con muchas líneas de código (stack trace) que cerraba la consola de golpe, haciendo fea la experiencia de uso. Se encapsulo todas las consultas SQL dentro de bloques try-catch en la capa DAO. Si salta una `SQLException`, la atrapamos y lanzamos una excepción propia creada por nosotros (por ejemplo, `StockInsuficienteException` o `ResourceNotFoundException`). En el menú de la consola atrapamos esa excepción propia y le mostramos un mensaje sencillo y claro al usuario.

- Al hacer muchas pruebas de consultas, se vio que, si se olvidaba de cerrar la conexión o las consultas SQL, la base de datos quedaba bloqueada o se llenaba de conexiones abiertas. Aplicamos la estructura try-with-resources en todos los DAOs. Al declarar la conexión entre los paréntesis del try (), Java se encarga de cerrarla automáticamente apenas termina la consulta, incluso aunque ocurra un error en el medio. Con esto se asegura de no dejar conexiones abiertas.

CAPTURAS DE PANTALLA

A continuación, un ejemplo de la funcionalidad del sistema:

- Inicio del menú principal

```
=====
      INICIANDO SISTEMA CON DIAGNOSTICO DE CONEXION
=====
Conexion exitosa a: jdbc:mysql://localhost:3306/food_store
Unidad de Persistencia: foodstorePU-mysql
=====

=====
      SISTEMA FOOD STORE - CONSOLA
=====
1. Gestion de Categorías
2. Gestion de Productos
3. Gestion de Usuarios
4. Gestion de Pedidos
5. Ver configuracion de conexion
0. Salir
Ingrese una opcion:
```

- Menú CRUD categorías, creamos una categoría. Podemos ver alguna de las validaciones

```

Que desea realizar?:
2

| LISTAR CATEGORIAS |

--- MENÚ CATEGORIAS ---

1) Listar
2) Crear
3) Editar
4) Eliminar (baja lógica)
0) Volver al menu principal

Que desea realizar?:

| DATOS INVÁLIDOS | La descripción de la categoría a crear no es válida |

| CREAR CATEGORIAS |

Nombre de la nueva categoría:
Repuestos
Descripción de la nueva categoría:
Repuestos de camiones y colectivos
| EXITO | | Categoría guardada correctamente |

```

- Intentamos editar un producto. Vemos algunas de las validaciones

```

--- MENÚ PRODUCTOS ---

1) Listar

2) Crear

3) Editar

4) Eliminar (baja lógica)

5) Listar por categoría

0) Volver al menú principal

Que desea realizar?:
3
| LISTAR PRODUCTOS POR CATEGORIA |

--- LIMPIEZA ---

| No hay productos asignados a esta categoría |

--- EMBUTIDOS ---

| Id del Producto: '3' |
| Nombre del Producto: 's' |
| Descripción del Producto: 'd' |
| Url de la imagen del Producto: 'd' |
| Precio del Producto: '$0.0' |
| Stock del Producto: '0' unidad/es |
| Disponibilidad del Producto: 'Disponible' |

--- HOGAR ---

| No hay productos asignados a esta categoría |

--- REPUESTOS ---

| No hay productos asignados a esta categoría |

| EDITAR PRODUCTOS |

| Dime el nombre o id del producto a editar(Indiferentemente de las mayúsculas o minúsculas):

| Dime el nombre o id del producto a editar(Indiferentemente de las mayúsculas o minúsculas):
hol
No hay un producto existente para el valor ingresado

| Dime el nombre o id del producto a editar(Indiferentemente de las mayúsculas o minúsculas):
3

| RESULTADOS DEL PRODUCTO BUSCADO |

| Id del Producto: '3' |
| Nombre del Producto: 's' |
| Descripción del Producto: 'd' |
| Url de la imagen del Producto: 'd' |
| Precio del Producto: '$0.0' |
| Stock del Producto: '0' unidad/es |
| Disponibilidad del Producto: 'Disponible' |

| EDITAR PRODUCTO |

Nombre del producto: motor
Descripción del producto: motor del camion
Url de la imagen del producto:
Precio: -1
Stock inicial: 4
|DATOS INVÁLIDOS| | El precio del producto no es válido |

```

```

| EDITAR PRODUCTO |

Nombre del producto: Motor
Descripcion del producto: Motor del Camion
Url de la imagen del producto: motor.jpg
Precio: 5000
Stock inicial: 40
| EXITO | | Producto actualizado correctamente |

| LISTAR PRODUCTOS POR CATEGORIA |

--- LIMPIEZA ---

| No hay productos asignados a esta categoria |

--- EMBUTIDOS ---

| Id del Producto: '3' |
| Nombre del Producto: 'Motor' |
| Descripcion del Producto: 'Motor del Camion' |
| Url de la imagen del Producto: 'motor.jpg' |
| Precio del Producto: '$5000.0' |
| Stock del Producto: '40' unidad/es |
| Disponibilidad del Producto: 'Disponible' |

--- HOGAR ---

| No hay productos asignados a esta categoria |

--- REPUESTOS ---

| No hay productos asignados a esta categoria |

```

- Creamos un usuario e intentamos eliminarlo

```

| CREAR USUARIOS |

Nombre del usuario: matias
Apellido del usuario: ferreyra
Email del usuario: matiasferreyra
Telefono del usuario: 3443244
Contraseña del usuario: jolaasl
Eres admin?(si/no): no

| Desea guardar el usuario?(si/no): si
|DATOS INVALIDOS| | El formato del correo electronico es invalido. |
| CREAR USUARIOS |

Nombre del usuario: Matias
Apellido del usuario: Ferreyra
Email del usuario: matias06ferreyra@gmail.com
Telefono del usuario: 54950495984
Contraseña del usuario: hola123
Eres admin?(si/no): no

| Desea guardar el usuario?(si/no): si
| EXITO |
| Usuario guardado correctamente |

| LISTAR USUARIOS |

| Id del Usuario: '1' |
| Nombre del Usuario: 'Matias Ferreyra' |
| Telefono del Usuario: '54950495984' |
| Email del Usuario: 'matias06ferreyra@gmail.com' |
| Rol del Usuario: 'USUARIO' |
| Vigencia del Usuario: 'Vigente' |

| ELIMINAR USUARIOS |

| Dime el id del usuario a eliminar: 2
| Usuario no encontrado o no existente |

| ELIMINAR USUARIOS |

| Dime el id del usuario a eliminar: 1

| Id del Usuario: '1' |
| Nombre del Usuario: 'Matias Ferreyra' |
| Telefono del Usuario: '54950495984' |
| Email del Usuario: 'matias06ferreyra@gmail.com' |
| Rol del Usuario: 'USUARIO' |
| Vigencia del Usuario: 'Vigente' |

| Desea eliminar el usuario?(si/no): si
| EXITO |
| Usuario eliminado correctamente |

```

- Creamos un pedido, y lo confirmamos. Viendo algunas de sus validaciones

```

| USUARIO 'mai' | ID: 2 |

| Ingrese ID del usuario:
2

| PEDIDOS DEL USUARIO 'mai' |

| No hay pedidos guardados en el usuario |

| CATEGORIAS |

| Id de la Categoría: '1' |
| Nombre de la Categoría: 'limpieza' |
| Descripción de la Categoría: 'hola ma' |
| Vigencia de la Categoría: 'Disponible' |

| Id de la Categoría: '2' |
| Nombre de la Categoría: 'Embutidos' |
| Descripción de la Categoría: 'Fiambres y quesos al por mayor' |
| Vigencia de la Categoría: 'Disponible' |

| Id de la Categoría: '5' |
| Nombre de la Categoría: 'Hogar' |
| Descripción de la Categoría: 'Articulos para el hogar' |
| Vigencia de la Categoría: 'Disponible' |

| Id de la Categoría: '6' |
| Nombre de la Categoría: 'Repuestos' |
| Descripción de la Categoría: 'Repuestos de camiones y colectivos' |
| Vigencia de la Categoría: 'Disponible' |

| Ingrese id o nombre de la categoría a filtrar:
6

| PRODUCTOS DE LA CATEGORIA 'Repuestos'
| No hay productos asociados a esta categoría |

```

```

| Dime el id del producto a agregar('salir' para terminar de agregar):
3
| PRODUCTO 'Motor' |

| Id del Producto: '3' |
| Nombre del Producto: 'Motor' |
| Descripcion del Producto: 'Motor del Camion' |
| Url de la imagen del Producto: 'motor.jpg' |
| Precio del Producto: '$5000.0' |
| Stock del Producto: '40' unidad/es |
| Disponibilidad del Producto: 'Disponible' |

Dime la cantidad a agregar:
10

| Dime el id del producto a agregar('salir' para terminar de agregar):
salir

| FORMAS DE PAGO |

| 'TARJETA' |

| 'TRANSFERENCIA' |

| 'EFECTIVO' |

| Como deseas pagar: tarjeta

==INFORMACION DEL PEDIDO ==
|Producto solicitado: Motor
|Subtotal del Producto solicitado: 50000.0
|Cantidad del Producto solicitado: 10

==PEDIDO ==
Total a pagar: 50000.0
Forma de pago: TARJETA

| Desea crear el pedido?(si/no):
si
| EXITO | | Pedido creado correctamente |

```

```

| PEDIDO N♦ '1' |

| Id del Pedido: '1' |
| Fecha de creacion del Pedido: '2026-10-01' |
| Estado del Pedido: 'PENDIENTE' |
| Total del Pedido: '250000.0' |
| Forma de pago: 'TRANSFERENCIA' |

| DETALLES DEL PEDIDO |

| Id del Detalle: '1' |
| Subtotal del Detalle: '250000.0' |
| Cantidad del Detalle: '50' |
| Nombre del Producto del Detalle: 'Motor' |

| Ingrese el id del pedido a confirmar: |
1

| PEDIDO N♦ '1' |

| Id del Pedido: '1' |
| Fecha de creacion del Pedido: '2026-10-01' |
| Estado del Pedido: 'PENDIENTE' |
| Total del Pedido: '250000.0' |
| Forma de pago: 'TRANSFERENCIA' |

| DETALLES DEL PEDIDO |

| Id del Detalle: '1' |
| Subtotal del Detalle: '250000.0' |
| Cantidad del Detalle: '50' |
| Nombre del Producto del Detalle: 'Motor' |

| Desea cambiar la forma de pago del pedido?(si/no): |
no

| Desea confirmar el pedido?(si/no): |
si
| STOCK INSUFICIENTE | | La cantidad solicitada del producto no puede ser mayor a la almacenada en memoria |

```

```

| PEDIDOS PENDIENTES DEL USUARIO 'mai' |

| PEDIDO N 0 |

| Id del Pedido: '0' |
| Fecha de creacion del Pedido: '2026-10-01' |
| Estado del Pedido: 'PENDIENTE' |
| Total del Pedido: '50000.0' |
| Forma de pago: 'TARJETA' |

| DETALLES DEL PEDIDO |

| Id del Detalle: '0' |
| Subtotal del Detalle: '50000.0' |
| Cantidad del Detalle: '10' |
| Nombre del Producto del Detalle: 'Motor' |

| Ingrese el id del pedido a confirmar: |
0

| PEDIDO N 0 |

| Id del Pedido: '0' |
| Fecha de creacion del Pedido: '2026-10-01' |
| Estado del Pedido: 'PENDIENTE' |
| Total del Pedido: '50000.0' |
| Forma de pago: 'TARJETA' |

| DETALLES DEL PEDIDO |

| Id del Detalle: '0' |
| Subtotal del Detalle: '50000.0' |
| Cantidad del Detalle: '10' |
| Nombre del Producto del Detalle: 'Motor' |

| Desea cambiar la forma de pago del pedido?(si/no): |
no

| Desea confirmar el pedido?(si/no): |
si

| EXITO | | Pedido confirmado correctamente |

| USUARIO 'mai' | ID: 1 |

| PEDIDOS VIGENTES |

| PEDIDO N 0 |

| Id del Pedido: '0' |
| Fecha de creacion del Pedido: '2026-10-01' |
| Estado del Pedido: 'TERMINADO' |
| Total del Pedido: '50000.0' |
| Forma de pago: 'TARJETA' |

| DETALLES DEL PEDIDO |

| Id del Detalle: '0' |
| Subtotal del Detalle: '50000.0' |
| Cantidad del Detalle: '10' |
| Nombre del Producto del Detalle: 'Motor' |

| PEDIDO N 1 |

| Id del Pedido: '1' |
| Fecha de creacion del Pedido: '2026-10-01' |
| Estado del Pedido: 'CONFIRMADO' |
| Total del Pedido: '250000.0' |
| Forma de pago: 'TRANSFERENCIA' |

| DETALLES DEL PEDIDO |

| Id del Detalle: '1' |
| Subtotal del Detalle: '250000.0' |
| Cantidad del Detalle: '50' |
| Nombre del Producto del Detalle: 'Motor' |

```

BIBLIOGRAFIA Y WEBGRAFIA

- Oracle Corporation (2023). Oracle Help Center
- Oracle Corporation (2023). Java Documentation APIs
- MySQL Developer Zone
- Jakarta EE Specification Project
- Guide to JDBC Connection Pooling, Transactions and Exception Handling. Baeldung.